SignalTrue — Product Logic and System Overview

Generated from codebase on 2026-05-02

Table of Contents

% SignalTrue — Product Logic and System Overview
% SignalTrue Product Logic and System Overview
% Generated from codebase

SignalTrue Product Logic and System Overview

Generated from the repository at / (branch: main) on 2026-05-02. This document is a code-first, evidence-based technical product overview for founders, developers, investors, pilot clients, and technical advisors. It references actual files and functions in the repository; where logic is missing or mocked, that is stated explicitly.

Table of contents

1. Executive Summary

What is SignalTrue (from code):

SignalTrue is implemented as a Behavioral Drift Intelligence platform that passively ingests metadata from calendar, messaging, and other workplace integrations and computes team-level indices and risk scores. The codebase centers on a Behavioral Drift Index (BDI) and weekly risk calculations (overload, execution, retention_strain). Key implementation files:

- Backend entry: `backend/server.js`
- BDI implementation: `backend/services/bdiService.js` and model `backend/models/behavioralDriftIndex.js`
- Risk scoring: `backend/services/riskCalculationService.js` and models `backend/models/riskWeekly.js`, `backend/models/riskDriver.js`
- Daily metrics storage: `backend/models/metricsDaily.js`

Core idea (explicit in code): use passive metadata (meeting duration, after-hours rates, response latency, collaboration breadth, etc.) to detect early deviations from a team-specific baseline. The system computes weekly scores, tracks drivers, and surfaces playbooks and AI-generated recommendations.

Privacy positioning (from code and routes): the AI Copilot endpoints require a privacy_mode of `metadata_only` (see `backend/routes/aiCopilot.js`). The code also includes baseline confidence and data-quality checks in `backend/utils/baselineCalculations.js` (e.g., `meetsDataQualityRequirements`) and enforces minimum group-size checks in those utilities (default minGroupSize=8), although some endpoints and UI components do not enforce minimums consistently.

Important clarification: This document reflects implemented logic in code. Where the code contains placeholders, mocks, or TODOs, those are called out explicitly in later sections.

2. Product Positioning

Category: Behavioral Drift Intelligence / Team Health Analytics (code manifests this through BDI, RiskWeekly, Signals, Playbooks).

Target users and buyer personas (implied and present in code/UI):
- CEO / Executive (executive brief PDF & weekly brief logic in `backend/services/weeklyBriefService.js`)
- COO / Operations (cost-of-drift routes `backend/routes/costOfDrift.js`, ROI models)
- HR leader / People analytics (playbooks, AI copilot playbook endpoints, `backend/routes/aiCopilot.js`)
- Team Lead / Manager (manager prompts, `generateManagerPrompts` in `weeklyBriefService.js`)
- People analytics leader / Data teams (benchmarks routes `backend/routes/benchmarks.js`)
- Hybrid/remote organizations (integrations for Slack/Google/Outlook are present)

Primary use cases (supported by code):
- Early detection of capacity and coordination problems via BDI and weekly risk scores
- Executive weekly briefs and alerts generated from signals and playbooks
- AI-assisted explanation and recommended actions through Copilot endpoints
- Integration of calendar and messaging metadata for signal extraction

Main value proposition (code-backed): give team- and org-level early warnings about overload, coordination drag, and cohesion drift using passive metadata, plus suggested playbooks and manager prompts.

3. Core Product Concept

System flow (implementation mapping):

Connect integrations (Slack, Google Calendar, Microsoft Outlook/Teams) — see `backend/services/integrationPullService.js`, `backend/services/slackService.js`, `backend/services/calendarService.js`, and routes `backend/routes/slackRoutes.js`, `backend/routes/calendarRoutes.js`.

Sync metadata into `IntegrationMetricsDaily`, `MetricsDaily`, and `WorkEvent` collections (`backend/models/integrationMetricsDaily.js`, `backend/models/metricsDaily.js`, `backend/models/workEvent.js`).

Calculate weekly/daily metrics and baselines with utilities in `backend/utils/baselineCalculations.js` and services in `backend/services/*` (BDI, riskCalculationService).

Detect deviations and build signals (`backend/models/signal.js`, `backend/routes/signals.js`, `backend/services/signalGenerationService.js` exists in the codebase list).

Create `BehavioralDriftIndex` records (`backend/services/bdiService.js` triggers pre-save hooks that compute drift state) and `RiskWeekly` entries (`riskCalculationService.js`).

Populate dashboards and weekly briefs (`backend/services/weeklyBriefService.js`, frontend components `src/components/BDIDashboard.tsx`, `src/components/DriftAlertCard.tsx`).

Generate recommendations and playbooks: a mix of rule-based playbooks (`backend/services/actionPlaybookService.js` referenced) and AI-assisted text via the Copilot service (`backend/routes/aiCopilot.js` calling `backend/services/aiCopilotService.js` and `backend/utils/aiProvider.js`).

Simple diagram (text):

Integration connectors -> ingestion services -> raw events / metrics (MetricsDaily, IntegrationMetricsDaily) -> baseline calculations -> BDI & RiskWeekly -> Signals & RiskDrivers -> Weekly briefs, dashboards, AI Copilot -> Playbooks / Actions / Exports

4. Data Sources and Integrations

Implemented / present in code (explicit files and services):
- Slack: `backend/services/slackService.js`, routes `backend/routes/slackRoutes.js`.
- Data collected: channel message counts, avg response delay, after-hours message counts; stored on Team.slackSignals and MetricsDaily fields (uniqueContacts, message counts). See `backend/models/team.js` and `backend/models/metricsDaily.js`.
- Content collected: code indicates metadata only (message counts, delays). Actual message text scrubbing is not visible in the code excerpts — some models exist for `chatLog.js` but most scoring relies on counts and timing.
- Implementation status: implemented (refresh functions exist), but message-level content usage is

unclear (there is a `chatLog.js` model — review needed if content is stored).

Google Chat: `backend/routes/googleChatRoutes.js` and team.googleChatSignals fields are present.

- Data collected: message counts, thread depth, after-hours counts; stored in Team.googleChatSignals.
- Status: route files exist; likely implemented analogously to Slack.

Google Calendar / Google Meet: `backend/services/calendarService.js`, `backend/routes/calendarRoutes.js`, `src/components/GoogleCalendarConnect.js`.

- Data collected: meeting hours, meeting durations, after-hours meetings, focus hours, back-to-back counts — stored in `MetricsDaily` (meetingHoursWeek, meetingLoadIndex) and `Team.calendarSignals`.
- Status: implemented (refresh scripts, cron jobs scheduled in `server.js`), but precise OAuth flow and Google service account usage depends on environment variables (e.g., `GOOGLE_SERVICE_ACCOUNT`).

Outlook / Microsoft Teams: evidence in models and event counts aggregated in `weeklyBriefService.js` (e.g., 'microsoft-outlook', 'microsoft-teams'). There are models like `backend/models/outlookSignal.js`.

- Data collected: meeting counts, messages; stored as work events/integration metrics.
- Status: implemented hooks/aggregation code present; actual adapter code location: `backend/services/coreIntegrationAdapters.js`.

Email (Gmail / Outlook): recommended by the Copilot `what-to-measure` logic; IntegrationMetricsDaily includes after-hours email metrics; routes and services reference Gmail/Outlook sources. Actual connectors are present in integration services but may require secrets.

Task/Project tools (Jira, Asana): referenced as recommended connectors by Copilot (`what-to-measure`), but I did not find a full Jira or Asana adapter in the scanned files; likely planned or partially implemented.

HRIS / People systems: minimal references in models (team metadata) but no full HRIS sync code found in the inspected files.

CSV / manual upload: not prominent — exports and admin tools exist (`backend/routes/exportRoutes.js`, `backend/routes/adminExport.js`) but I did not find a generic CSV upload ingestion for signals.

For each integration, the code emphasizes metadata-only collection. The `aiCopilot` endpoints explicitly enforce `privacy_mode: 'metadata_only'` for payloads sent to AI (`backend/routes/aiCopilot.js`).

Where integrations are mocked/partial: the provider adapter `backend/utils/aiProvider.js` includes a fallback mock and supports OpenAI and Anthropic. Some routes check for environment variables and will be disabled if tokens are missing (see `backend/server.js` warnings). Several scripts exist to refresh and debug integrations (`backend/scripts/*.js`), indicating active development and operational tooling.

Storage: integration-derived metrics land in `IntegrationMetricsDaily`, `MetricsDaily`, `WorkEvent`, Team.*signals and are stored in MongoDB (connection in `backend/server.js`).

5. Privacy and Data Protection Logic

What the code does explicitly:

- Enforced metadata-only payloads for AI endpoints: `backend/routes/aiCopilot.js` validates `policies.privacy_mode === 'metadata_only'`.
- Baseline and data quality utilities include a minimum group-size check (`meetsDataQualityRequirements` in `backend/utils/baselineCalculations.js`) with a default `minGroupSize = 8`. This is a localized privacy safeguard.
- `backend/server.js` mounts a consent-audit middleware (`middleware/consentAudit.js`) on `/api/consent-audit` and passes it at times when consent is required: the server uses `auditConsent` middleware for consent-protected routes.
- AI usage is tracked to a local JSON file `backend/ai-usage.json` via `backend/utils/aiUsage.js`.

What is unclear or missing (risks):

- Minimum team-size enforcement in all UI and API surfaces is inconsistent. The baseline util has `minGroupSize=8` but other endpoints and UI components do not uniformly check team size

before returning dashboards or BDI — this is a gap.
- I did not find a global config enforcing anonymization or permanent deletion of raw event payloads; models such as `chatLog.js` and `documentChunk.js` exist — they may store text. Where raw content is stored is not obvious and must be audited carefully.
- No explicit retention policy implementation was found (e.g., automatic purge older than N days). There is a script `backend/utils/purgeExpiredData.js`, but I did not find a scheduled job wiring it in `server.js` cron list.
- GDPR/DSAR handling endpoints are not obvious in the scanned routes; `consentAudit` exists but full DSAR handling (export/delete user personal data) isn't visible in the inspected files.

Recommendations (short):
- Enforce minimum-sample-size at API gates that return team-level analytics and in front-end components. Default should be 5+ (strong recommendation) or 8+ (current default in utils). Document and implement suppression UI for insufficient samples.
- Audit any model that stores text (e.g., `chatLog.js`, `documentChunk.js`) and ensure they are either not storing message content or are encrypted and access-controlled.
- Implement explicit retention policies and DSAR endpoints.

6. Core Metrics (what the code calculates)

The codebase defines a number of metrics and indexes. Below are the principal ones found, with the file references where the metric is defined or used.

Primary stored metrics (models):
- MetricsDaily (`backend/models/metricsDaily.js`):
- `meetingHoursWeek`, `meetingLoadIndex`, `afterHoursRate`, `responseMedianMins`, `responseLatencyTrend`, `sentimentAvg`, `uniqueContacts`, `focusTimeRatio`, `energyIndex`.

Team current signals (`backend/models/team.js`):
- `slackSignals` (messageCount, avgResponseDelayHours), `googleChatSignals` (messageCount, avgResponseDelayHours, afterHoursPercentage), `calendarSignals` (meetingHoursWeek, afterHoursMeetings, focusHoursWeek, focusToMeetingRatio).

BehavioralDriftIndex (`backend/models/behavioralDriftIndex.js`):
- Signals: meetingLoad (hours/week), afterHoursActivity (%), responseTime (hours), asyncParticipation (message count), focusTime (hours/week), collaborationBreadth (unique collaborators). Derived fields: `driftScore` (0-100), `state` (Stable, Early Drift, Developing Drift, Critical Drift), `topDrivers`.

RiskWeekly (`backend/models/riskWeekly.js`): weekly `score` (0-100), `band` (green/yellow/red) for risk types: `overload`, `execution`, `retention_strain`.

RiskDriver (`backend/models/riskDriver.js`): traceability of which metrics contributed to weekly risk scores.

Derived signals and dashboards rely on additional computed metrics present in `IntegrationMetricsDaily` and `WorkEvent` (used by `weeklyBriefService.js`). The weekly brief derives averages for `meetingCount7d`, `meetingDurationTotalHours7d`, `backToBackMeetingBlocks`, `messageCount7d`, `afterHoursMessageRatio`, `focusTimeAvailabilityHours`, `calendarFragmentationScore`, etc. These are aggregated into observations and recommendations.

Status of metrics: mostly implemented (models and services exist). Calculation code exists for many scores (BDI pre-save hook, `riskCalculationService.js` formulas). Some metrics (e.g., `energyIndex`) mention auto-tuned weights but the auto-tuning implementation is not obvious in the files read.

7. Calculation Logic (evidence-based)

This section lists the formulas or calculation patterns explicitly found in code. Where the code does not contain explicit formulas, that is stated.

Overload Risk (`backend/services/riskCalculationService.js` — function `calculateOverloadRisk`)
- Inputs (mapped to `MetricsDaily` fields via `METRIC_FIELD_MAP`):
- after_hours_activity -> `afterHoursRate`
- meeting_load -> `meetingLoadIndex`
- back_to_back_meetings -> `meetingHoursWeek` (used as proxy)
- focus_time -> `focusTimeRatio` (higher is better)

Formula (explicit in code):

overload_risk = 0.35 * deviation(after_hours_activity) + 0.30 * deviation(meeting_load) + 0.20 * deviation(back_to_back_meetings) + 0.15 * adjusted_deviation(focus_time)

- deviation(...) is normalized and clamped to [-1, +1] using function `calculateDeviation(currentValue, baselineMean, isHigherBetter)` in the same file. For metrics where higher is better (focus_time), the deviation is inverted.
- Only positive deviations contribute to risk (i.e., Math.max(0, deviation) applied before weighting).
- Final score scaled to 0-100 by Math.round(score * 100).
- Risk band mapping: score < 35 => 'green', <65 => 'yellow', otherwise 'red'.

Persistence and drivers: the service writes `RiskWeekly` and `RiskDriver` entries and returns drivers where deviation > 0.1.

Execution Risk (`calculateExecutionRisk` in same file)

- Inputs: response_time, participation_drift, meeting_fragmentation, focus_time.
- Weights: 0.30, 0.25, 0.25, 0.20 respectively.
- Calculation pattern identical to Overload Risk: normalized deviations, positive contributions only, scaled to 0-100.

Retention Strain Risk (`calculateRetentionStrainRisk`)

- Inputs: 3-week trend slopes for after_hours_activity, meeting_load, response_time.
- Weights: 0.40, 0.30, 0.30.
- Trend slope is computed with `calculateTrendSlope(metricsHistory, metricKey)` performing a simple linear regression then normalizing by mean (slope / meanY). Contributions are max(0, slope) * weight, aggregated and scaled to 0-100.

Behavioral Drift Index (BDI) (`backend/models/behavioralDriftIndex.js` pre-save hook and `backend/services/bdiService.js`)

- BDI inputs: meetingLoad, afterHoursActivity, responseTime, asyncParticipation, focusTime, collaborationBreadth.
- Baseline: first 30 days or existing baseline saved in BDI records.
- Thresholds: stored in document `thresholds` with defaults (meetingLoad: 20% change, afterHoursActivity: 30%, responseTime: 25%, asyncParticipation: 20%, focusTime: 20%, collaborationBreadth: 25%).
- Logic in pre-save hook:
- For each signal, calculate percentChange = ((current - baseline)/baseline)*100.
- If |percentChange| > threshold, mark `deviating` true and determine direction (for some signals higher is negative, for others lower is negative).
- Count `deviatingSignalsCount` and `negativeSignalsCount`.
- `topDrivers` is the top 3 signals by absolute percentChange.
- `driftScore` is computed as Math.min(Math.round((negativeCount / 6) * 100), 100).
- State mapping:
- negativeCount 0-1 => Stable
- 2 => Early Drift
- 3-4 => Developing Drift
- 5-6 => Critical Drift

Summary generated and stored in `summary` field.

Baseline statistics (`backend/utils/baselineCalculations.js`)

- Baseline window and robust statistics are implemented: mean, median, std, MAD, p25, p75.
- Confidence scoring: depends on data coverage and sample size; `calculateBaselineConfidence` produces 0..1.
- Robust Z-scores via MAD provided (`calculateRobustZScore`).

What is missing (gaps):

- The BDI driftScore is a coarse mapping from count of negative signals; no continuous weighting by magnitude is present in the BDI pre-save hook (the top drivers include change magnitude, but the score is purely count-based). If the product needs finer-grained scoring, that is a gap.
- Some metrics like `energyIndex` claim auto-tuning; I did not find the auto-tune implementation in the inspected files.

8. Baselines and Drift Detection

Baseline approach (explicit in code):
- `bdiService.getOrEstablishBaseline` uses the first 30 days as the baseline and stores baseline values in the `BehavioralDriftIndex.baseline` object (see `backend/services/bdiService.js`).
- `backend/utils/baselineCalculations.js` defines robust baseline statistics and suggests using a window of `BASELINE_WEEKS` (from config) and robust metrics (median, MAD, percentiles). The constants are configurable via `config/signalTemplates.js` (not fully inspected here).

Drift detection approaches used:
- Percent-change vs baseline with per-signal thresholds (BDI thresholds in model defaults). This is the primary method for BDI.
- Robust z-scores / MAD are available for anomaly detection but I didn't find a direct hook that uses z-scores for decisioning in BDI (the utilities exist for building more robust detectors).
- Trend-slope detection for retention strain (3-week linear regression slope normalized by mean in `calculateTrendSlope`).

Comparisons used in code:
- Team-specific baseline (BDI): first 30 days stored in each team/BDI record.
- Rolling recent windows: many services use last 7 days and 3-week windows (e.g., getCurrentMetrics uses last 7 days; retention uses 3 weeks; weekly briefs use 1-week vs last-week vs 6-week averages).
- Organization/industry benchmarks: `backend/models/IndustryBenchmark.js` and `backend/routes/benchmarks.js` are present for external comparisons, but the linkage into public scoring is partial.

If you need explicit anomaly detection with thresholds (e.g., z-score>2), the codebase has utilities but some decision rules remain heuristics (percent thresholds in BDI, weightings in riskCalculationService). The baseline calculations library supports confidence scoring and time-series banding used in UI time-series builders (`buildTimeSeries`).

9. Risk Classification

Risk types and bands (explicit):
- Risk types: `overload`, `execution`, `retention_strain` stored in `RiskWeekly`.
- Score: 0..100 (calculated by services). Banding: <35 green, 35-64 yellow, >=65 red (see `getRiskBand` in `riskCalculationService.js`).

Team state mapping (in `determineTeamState`):
- `healthy`: all risks < 35
- `strained`: any risk >= 35
- `overloaded`: overload_risk >= 65
- `breaking`: execution_risk >= 65 for 2+ weeks

Triggers and UI effects (from code references):
- Weekly risk write: `RiskWeekly` entries are saved with explanation text and drivers.
- `RiskDriver` entries provide explainability for which metrics contributed.
- `BehavioralDriftIndex` maps to human states (Stable / Early Drift / Developing Drift / Critical Drift) and top drivers.

Model type: rule-based + weighted scoring (riskCalculationService uses explicit weights and linear combinations). AI may be used for explanation and playbooks, but not for core numeric score computation.

10. Recommendations Logic

Where recommendations come from (code):
- Rule-based playbooks: `backend/services/actionPlaybookService.js` and `backend/models/driftPlaybook.js` (playbooks exist and are returned by Copilot 'playbooks' endpoints).
- AI-assisted templates: `backend/routes/aiCopilot.js` delegates to `backend/services/aiCopilotService.js` which uses `backend/utils/aiProvider.js` to call OpenAI or Anthropic. Prompts live in `backend/prompts/` (e.g., `weeklyAiPrompt_v1.json`, `monthlyStrategicAiPrompt_v1.json`).

Behavior in practice (from `aiCopilot.js`):
- The `/api/ai/copilot` endpoint builds or accepts a signals payload and then calls `generateCopilotResponse(payload)`.
- If no signals exist, the endpoint returns a safe no-op response (no diagnosis language, metadata_only_confirmed). This demonstrates privacy-safety guard rails.

- Playbooks and actions can also be selected locally without AI using `selectActions(...)` — a purely rule/template-based mapping.

Recommendation artifacts and storage:
- Recommended actions are returned in API responses; there are models for `driftPlaybook` and `action` that can be saved and associated with BDIs.
- Teams have `recommendedAction` and `playbook` fields on `Team` objects which can store generated playbooks (e.g., `team.playbook` in `team.js`).

User interactions:
- The Copilot API supports feedback (`/copilot/feedback`) but currently only logs feedback; persistence is TODO.
- Actions/playbooks selection endpoints exist and are used by the front-end components (`src/components/PlaybookRecommendations.js`).

Rule vs AI: core recommendations are a hybrid: rule/template playbooks plus optional AI-generated text. When AI is used, the code enforces metadata-only policy and tracks usage.

11. Reports and Insights

Reports implemented or present in code:
- Weekly Briefs: `backend/services/weeklyBriefService.js` builds an HTML-like brief with observations, risks, and recommended actions. It references `WeekContext`, `CategoryKingSignal`, `Signal`, and `BehavioralDriftIndex`.
- Weekly/Monthly reports: models `weeklyReport.js` and `monthlyReport.js` exist; front-end pages `src/pages/DriftReport.tsx`, `src/pages/MonthlyReport.tsx` render report UI.
- Executive summary / CEO summary: `src/pages/CeoSummary.tsx` and model `backend/models/ceoSummary.js` exist.
- PDF export: `backend/routes/reports.js` and `backend/routes/exportRoutes.js` exist; the generation flow is present but I did not verify an actual PDF generator dependency on the server — weeklyBriefService builds HTML and uses nodemailer/resend to email briefs (`transporter` configured). The code includes logic to generate message templates and manager prompts.

Who receives reports: the weeklyBriefService constructs briefs per organization and per team; recipients are determined by Organization/User settings in `Organization` and `User` models (not fully enumerated in the scanned files).

Frequency: weekly briefs (1-week windows) are explicit; monthly strategic prompts exist (`prompts/monthlyStrategicAiPrompt_v1.json`). Export routes exist for scheduled or on-demand usage.

Dashboard insights vs PDF reports: dashboards show live or recent metrics via API endpoints (BDI, RiskWeekly, MetricsDaily). Weekly briefs are narrative summaries produced by `weeklyBriefService.js`. PDF export plumbing exists but may depend on external rendering options; I did not find a simple single-function PDF renderer in the backend code — weekly briefs prepare HTML and can be emailed.

12. Dashboard and User Interface

Main frontend screens (files are present under `src/pages` and `src/components`):
- Public marketing: `src/pages/Index.tsx`, `src/pages/Product.tsx`, `src/pages/Pricing.tsx`.
- Login/Onboarding: `src/pages/Login.tsx`, `src/pages/Register.tsx`, `src/pages/AcceptInvitation.tsx`, `src/components/onboarding/*`.
- Integrations: `src/pages/IntegrationsPage.tsx`, components `GoogleCalendarConnect.js`, `GoogleChatConnect.js`.
- Dashboard / App: `src/pages/app/Overview.js`, `src/pages/app/ExecutiveSummary.js`, `src/pages/app/Insights.js`, `src/components/Dashboard.js`, `src/components/BDIDashboard.tsx`, `src/components/DriftAlertCard.tsx`, `src/components/CoordinationLoadIndexCard.js`, `src/components/CapacityStatusCard.js`.
- Team analytics and reports: `src/pages/TeamAnalytics.tsx`, `src/pages/DriftReport.tsx`, `src/pages/MonthlyReport.tsx`.
- AI Copilot UI: `src/components/AICopilotPanel.js`.

Data shown and data flow (examples):
- BDIDashboard and Drift alert cards call backend endpoints under `/api/bdi`, `/api/signals`, `/api/insights`, `/api/ai/copilot` to fetch data. The UI components reference model fields like `driftScore`,

`state`, `topDrivers` and render playbook recommendations.
- Integration pages call `/api/calendar/refresh` and `/api/slack/refresh` routes.
- Some components are mock-friendly: `src/components/DashboardMockup.js` indicates demo data presentation for marketing pages.

Mocked vs live data: many app components use live API calls; however, the marketing/staging pages and demo overlays (`DashboardMockup.js`, Paywall overlays) contain static or mocked data for marketing. The front-end contains both protected routes (`ProtectedRoute.js`) and public pages.

Screenshots: I did not capture visual screenshots. The repository includes many React components; the UI is a mix of marketing pages and in-app dashboards. The Drift diagnostic, weekly brief, and CEO summary pages render narratives and metric cards.

13. Database Schema (key collections)

Below are the most important collections (Mongoose models) used by SignalTrue. I list fields and purpose briefly using the actual model filenames.

`MetricsDaily` (`backend/models/metricsDaily.js`)
- Purpose: store daily normalized metrics per team.
- Key columns: teamId, orgId, date, meetingHoursWeek, meetingLoadIndex, afterHoursRate, responseMedianMins, sentimentAvg, uniqueContacts, focusTimeRatio, energyIndex, energyWeights.
- Used by: risk calculations, BDI, weekly briefs.

`BehavioralDriftIndex` (`backend/models/behavioralDriftIndex.js`)
- Purpose: primary drift record for teams over a time window.
- Key fields: orgId, teamId, periodStart, periodEnd, signals (six signals), state (Stable..Critical), driftScore, topDrivers, baseline, thresholds, confidence, recommendedPlaybooks.
- Used by: dashboards, weekly brief, drift timeline.

`RiskWeekly` (`backend/models/riskWeekly.js`)
- Purpose: stores weekly risk scores by riskType.
- Fields: teamId, weekStart, riskType, score, band, confidence, explanation.
- Used by: dashboards, team state determination.

`RiskDriver` (`backend/models/riskDriver.js`)
- Purpose: trace which metrics contributed to weekly risk.
- Fields: teamId, weekStart, riskType, metricKey, contributionWeight, deviation, explanationText.

`Team` (`backend/models/team.js`)
- Purpose: team metadata and current signals.
- Key fields: name, orgId, bdi, slackSignals, calendarSignals, googleChatSignals, baseline, metadata (function, sizeBand, actualSize), recommendedAction.

`IntegrationMetricsDaily` (model exists; used heavily in weeklyBriefService)
- Purpose: aggregated daily metrics per integration (meetingCount7d, messageCount7d, afterHoursMessageRatio, etc.).

`WorkEvent` (`backend/models/workEvent.js`)
- Purpose: low-level event counts aggregated from connectors (messages, meetings, eventType).

`Signal`, `CategoryKingSignal` (`backend/models/signal.js`, `backend/models/categoryKingSignal.js`)
- Purpose: discrete signals detected and labeled (e.g., 'meeting-load-spike', 'after-hours-creep'). These are used to populate briefs and recommendations.

`DriftTimeline`, `DriftEvent` (`backend/models/driftTimeline.js`, `backend/models/driftEvent.js`)
- Purpose: track the lifecycle of detected drift and escalation events.

`User`, `Organization`, `Invite`, `ConsentAudit` etc. (standard auth & governance data).

Relationships: Teams belong to Organizations; MetricsDaily and other metrics reference teamId and orgId. RiskWeekly and BehavioralDriftIndex reference teamId and store period information.

14. API and Backend Logic

I list important endpoints with their role and implemented status (based on route presence and code). Authentication: many routes use `authenticateToken` middleware; some routes are public (drift diagnostic, assessment, chat).

Representative endpoints:

Other backend systems:

- Scheduled jobs/cron: `server.js` schedules Slack refresh (cron schedule in `server.js`), there are many scripts and scheduler services referenced (`weeklySchedulerService.js`, `integrationSyncScheduler.js`). The code indicates many scheduled processes but the exact schedule wiring lives in `server.js` and service files.
- Background workers: some services write risk records and drivers; insertion is synchronous in service functions (no separate worker queue). There are scripts to seed data and check integratons.
- AI calls: `backend/utils/aiProvider.js` wraps OpenAI and Anthropic; `backend/utils/aiUsage.js` records usage locally.

Status: Most endpoints exist and appear implemented; some features are gated on environment variables (e.g., AI keys, Google service account). Some admin/debug routes exist only for superadmin.

15. System Architecture

High-level components (from `server.js` and services):

- Frontend: React app in root `src/` (CRA) serving UI.
- Backend: Express server `backend/server.js` + route modules in `backend/routes/`.
- Database: MongoDB (Atlas or in-memory for tests). Mongoose models in `backend/models/`.
- Integrations: Slack, Google (Calendar & Chat), Microsoft (Outlook/Teams) adapters in `backend/services/` and `backend/routes/`.
- AI layer: provider adapters in `backend/utils/aiProvider.js`; Copilot service and prompts in `backend/prompts/`.
- Scoring engine: riskCalculationService, bdiService, baselineCalculations — synchronous JS services.
- Reporting engine: weeklyBriefService, reports routes, and email transport using nodemailer/Resend.
- Scheduler: Cron jobs in `server.js` and scheduler services in `backend/services`.

Data flow example (concrete):

Slack messages aggregated by `slackService.refreshAllTeamsFromSlack()` -> populates `WorkEvent` and updates `Team.slackSignals`.

Calendar sync via `calendarService.refreshTeamCalendar()` -> updates `Team.calendarSignals` and `MetricsDaily`.

`riskCalculationService.calculateOverloadRisk(teamId)` reads recent `MetricsDaily` and `Baseline` and writes `RiskWeekly` and `RiskDriver` documents.

`bdiService.calculateBDI(teamId)` creates `BehavioralDriftIndex` entries; pre-save hook computes `driftScore` and `state`.

Frontend calls `/api/bdi` and `/api/ai/copilot` to render dashboard and to show suggested actions.

16. AI Usage

What the code does:

- AI providers supported: OpenAI and Anthropic (`backend/utils/aiProvider.js`). There is a fallback mock adapter when keys are missing.
- Prompts are stored in `backend/prompts/` (e.g., `weeklyAiPrompt_v1.json`). AI is used to synthesize signals into narrative text and to generate message templates and playbooks.
- AI input: Copilot enforces `privacy_mode: 'metadata_only'` and the `aiCopilot` endpoints validate this; the `generateCopilotResponse` service builds a payload from signals metadata and connector coverage (see `backend/routes/aiCopilot.js` and services). The code attempts to avoid sending raw message content.
- AI outputs: summaries, message templates, recommended actions, playbooks. Usage is logged by `backend/utils/aiUsage.js` to `backend/ai-usage.json`.

Risk assessment from code:

- The code seeks to restrict AI inputs to metadata, but presence of models like `chatLog.js` and `documentChunk.js` indicates potential for storing content; if content is sent to AI anywhere, that is

a compliance risk — I didn't find direct code that sends full message text to the AI provider, but that requires a manual audit of integration adapters.

Fallbacks: when AI provider keys are not set, the `aiProvider` returns a mock response; Copilot handlers have logic to return safe no-op outputs when no signals are present.

17. Current Implementation Reality (brutal)

Fully implemented (based on code presence and concrete functions):

- Core data ingestion patterns for calendar and messaging (calendarService, slackService, workEvent aggregation).
- Metrics storage models (`MetricsDaily`, `IntegrationMetricsDaily`) and BDI model with pre-save drift calculation.
- Weekly risk calculation engine with explicit formulas and weights (`riskCalculationService.js`).
- Weekly brief generation logic exists (`weeklyBriefService.js`) that builds narrative HTML and recommendations.
- AI Copilot endpoints with provider adapters and prompt files; metadata-only enforcement in the API layer.

Partially implemented / incomplete:

- Some connectors/adapters appear partial or gated by environment variables (Google service account, Slack tokens) — code exists but operational wiring depends on secrets.
- Auto-tuning and "energyIndex" auto-tune logic references are present but I could not find the training/auto-tuning implementation.
- Reports export to PDF: HTML generation exists, but a server-side PDF renderer is not obviously present in the code (weeklyBrief uses nodemailer and Resend to send HTML). PDF export endpoints exist but may rely on external rendering services.
- Some admin/debug routes are present but not necessarily production-hardened (many scripts and manual refresh endpoints).

Mocked / demo-only:

- Marketing/dashboard mockups: `src/components/DashboardMockup.js` and demo overlays present — these present static data for marketing and demo flows.
- `aiProvider` fallback and `ai-usage` local file logging are mock-friendly when API keys are missing.

Missing but important:

- Uniform enforcement of minimum team size prior to returning team-level analytics (the baseline util has checks but many API endpoints and UI components do not consistently use it).
- Explicit retention policy and DSAR endpoints (scripts exist but not a complete DSAR flow).
- Audit trail and access control for sensitive models that may contain content (e.g., `chatLog`, `documentChunk`).
- Full task management integrations (Jira/Asana) appear recommended but not fully present.

Technical risks:

- Potential storage of message content in some models (`chatLog.js`, `documentChunk.js`) — needs audit.
- Reliance on environment variables for critical connectors without clear onboarding flow (server warns and exits if required env vars missing).
- No clear background job queue for heavy calculations — synchronous services may cause scaling concerns for large orgs.

18. Suggested Improvements (prioritized)

Priority 1: Must fix before pilot

- Enforce minimum team-size privacy gate at API and front-end layers (why: protects privacy and reduces false signals). Action: central middleware that checks team actualSize or mapped activeUsersCount and returns 204/insufficient-sample when below threshold.
- Audit and lock down any raw content storage (chatLog, documentChunk). If content is stored, either remove, encrypt, or strictly gate access and avoid sending to AI. Why: compliance and trust.
- Implement retention policy and DSAR endpoints. Why: legal compliance.
- Add integration coverage checks and clear onboarding flows for connectors so data coverage is visible to customers (the Copilot 'what-to-measure' logic hints at this but it should be explicit in the UI).

Priority 2: Should fix before paid customers
- Add unit tests and integration tests for all scoring functions (riskCalculationService, bdiService). Why: scoring reproducibility and auditability.
- Convert narrative HTML -> PDF path into a deterministic server-side renderer (e.g., headless Chromium via Puppeteer or a serverless render) and add a scheduled job for weekly PDF generation. Why: robust report exports.
- Move heavy calculations into background workers / job queue (BullMQ / Redis) and add monitoring. Why: scalability for large orgs.

Priority 3: Later improvements
- Implement robust auto-tuning for composite indices like `energyIndex` with versioned model artifacts.
- Add industry-benchmarking pipelines to compute normalized z-scores vs role/size benchmarks.
- Add richer explainability tracking and feedback loop persisting Copilot feedback for model improvements.

19. Recommended Metric Model (if not fully implemented)

This recommended model is explicit: it should be treated as a design proposal if not already present in code.

Recommended composite scores (proposal):

Capacity Drift Score (weight 40%): inputs — meeting load, focus time loss, after-hours activity, back-to-back meetings, weekend work.

Coordination Drag Score (weight 35%): inputs — response latency, cross-team delays, meeting fragmentation, unresolved collaboration loops, dependency concentration.

Cohesion Drift Score (weight 25%): inputs — communication concentration, weak-tie reduction, team silence, newcomer connection patterns, manager interaction rhythm.

Combine to Overall Team Drift Score = 0.4 * Capacity + 0.35 * Coordination + 0.25 * Cohesion.

Note: The code currently implements related concepts (overload, execution, retention_strain, BDI) but the proposal above is a cleaner unified model if desired. Marked as "Recommended model, not yet implemented unless found in code." (I found overlapping elements but not an exact match with these weights.)

20. Final Summary

What SignalTrue currently is (code-backed):
- A privacy-minded, signal-driven team health product that ingests calendar and messaging metadata, computes per-team metrics, derives a Behavioral Drift Index, and computes weekly risk scores using explicit, rule-based formulas.

What it already does well:
- Clear, implemented scoring pipelines for overload / execution / retention risks with traceable drivers (RiskDriver model).
- A defensible BDI model that uses baseline comparisons and per-signal thresholds.
- Integrated weekly brief generation with narrative templates and manager prompts.
- AI Copilot integration with metadata-only guard rails and local AI usage tracking.

What must be fixed before pilot customers:
- Enforce minimum team-size privacy gating consistently across APIs and UI.
- Audit and remove/secure any storage of raw message content (if present).
- Implement retention/DSAR endpoints and production-ready connector onboarding flows.

What must be fixed before paid enterprise customers:
- Harden scaling (background jobs, worker queues), add monitoring and SLAs.
- Provide deterministic report (PDF) generation and export pipeline.
- Add tests and reproducible scoring validation and documentation of all formulas.

Strategic value:
- The codebase already contains the essential ingredients of a behavioral drift product: ingestion, baseline computation, scoring, explainability (drivers), playbooks, and AI-assisted narrative. With prioritized privacy hardening and operationalization of integrations and reporting, the system is well-positioned for pilots and early enterprise adoption.

Appendix: Key files referenced (partial list)

- backend/server.js
- backend/services/riskCalculationService.js
- backend/services/bdiService.js
- backend/utils/baselineCalculations.js
- backend/utils/aiProvider.js
- backend/utils/aiUsage.js
- backend/services/weeklyBriefService.js
- backend/routes/aiCopilot.js
- backend/models/behavioralDriftIndex.js
- backend/models/metricsDaily.js
- backend/models/riskWeekly.js
- backend/models/riskDriver.js
- backend/models/team.js
- src/components/BDIDashboard.tsx
- src/pages/DriftReport.tsx

If you want, I can now:

- Generate a PDF from this Markdown file and place it at `SignalTrue_Product_Logic_and_System_Overview.pdf` in the repository (I'll attempt to run pandoc or another renderer). If the environment lacks a PDF renderer, I'll provide one-command instructions to produce the PDF locally.
- Produce a shorter executive one-pager or a slide deck from the same content.

Next step: attempt to convert the Markdown to PDF in the workspace. I'll try using `pandoc` and fallback to returning the Markdown if conversion tools are unavailable.